

## PLAN ESTRATÉGICO DE QA Y QUALITY ENGINEERING

ACME Corp - Reto Técnico QA Lead

Autor: Elvis Miguel Mieleles Guarin

Fecha: Enero 2025

Versión: 1.0

---

## TABLA DE CONTENIDOS

1. Executive Summary
2. Contexto y Diagnóstico
3. Estrategia de QA (1.1)
4. Implementación con Restricciones (1.2)
5. Escalabilidad Organizacional (1.4)
6. Roadmap 30/60/90 Días
7. Gestión del Cambio
8. Riesgos y Mitigaciones
9. KPIs y Métricas
10. Conclusión

---

## Resumen EJECUTIVO

### Contexto

ACME Corp desarrolla una plataforma de mensajería empresarial similar a WhatsApp, utilizada por miles de usuarios diariamente. La organización cuenta con:

- 60 desarrolladores distribuidos en 9 subequipos
- Sprints de 1 semana con deploys frecuentes
- Equipo inicial de QA: 2 personas
- 90 días para demostrar impacto inicial
- Resistencia moderada en algunos equipos
- Diferentes niveles de madurez en QA

### Problemas Identificados

- Regresiones frecuentes después de despliegues
- 61% de funcionalidades críticas sin cobertura automatizada
- Problemas de precisión y consistencia en pruebas
- Bajo nivel de automatización en algunos dominios
- Baja visibilidad sobre cobertura, métricas y pipelines
- Incidentes en producción frecuentes
- Falta de estrategia transversal de QA

Solución Propuesta

Estrategia de Quality Engineering basada en 5 pilares fundamentales:

- 1. Ownership compartido - QA Champions + Developers como owners de calidad
- 2. Testing pyramid optimizado - 60% unitarias, 30% API/integración, 10% E2E + 95% cobertura crítica
- 3. 4 agentes de IA como multiplicador de fuerza - Cobertura, Clean Code, Casos de uso, Métricas
- 4. Quality Gates automáticos en CI/CD - Validación progresiva basada en riesgo
- 5. Visibilidad continua - Dashboards y métricas por audiencia

Plan de Implementación

Fase 1 (0-30 días): Diagnóstico + baseline + 6-8 flujos críticos automatizados + Quick wins visibles

Fase 2 (31-60 días): 95% flujos críticos automatizados + Quality Gates activos en PR + Estabilización

Fase 3 (61-90 días): Cultura Quality Engineering + defect leakage <5% + Ownership compartido establecido

Impacto Esperado (90 días)

| Métrica                            | Baseline | Target 90 días | Mejora |
|------------------------------------|----------|----------------|--------|
| Cobertura funcionalidades críticas | 39%      | 95%            | +144%  |
| Defect leakage rate                | 18%      | <5%            | -72%   |
| Incidentes en producción           | Baseline | -50%           | -50%   |
| Deployment success rate            | Baseline | >95%           | +X%    |
| Flujos críticos protegidos         | 0/15     | 15/15          | 100%   |

## Escalabilidad

El modelo propuesto escala de 60 a 150 developers:

- Con presupuesto: Contratar +1 QA (100 devs), +3 QA adicionales (150 devs)
- Sin presupuesto: Priorización radical (solo 3-4 equipos Po) + IA como reemplazo parcial + Champions autónomos

## Inversión

- Costo IA: ~\$17/mes (OpenAI/Claude API + AWS Lambda)
- Costo QA adicional: \$XX según escala (opcional si hay presupuesto)
- ROI: 75% ahorro tiempo en diseño casos de prueba + 50% reducción incidentes

## Riesgo

Viabile con 2 QA hasta 100 developers con estrategia aprobada. Post-100 requiere decisión: contratar o aplicar modelo sin presupuesto (con concesiones en cobertura P1/P2).

## CONTEXTO Y DIAGNÓSTICO

### Escenario Actual

**ACME Corp es una organización con:**

| Parámetro                    | Valor                  |
|------------------------------|------------------------|
| Desarrolladores              | 60                     |
| Subequipos                   | 9                      |
| Equipo QA inicial            | 2 personas             |
| Sprint length                | 1 semana               |
| Frecuencia de deploys        | Alta                   |
| Plazo para demostrar impacto | 90 días                |
| Nivel de resistencia         | Moderado               |
| Madurez QA                   | Variable entre equipos |

## **Problemas Críticos Identificados**

### ***Calidad y Cobertura***

- ✗ Regresiones frecuentes después de despliegues
- ✗ 61% de funcionalidades críticas sin cobertura automatizada
- ✗ Problemas de precisión y consistencia en pruebas existentes
- ✗ Bajo nivel de automatización en algunos dominios
- ✗ Suites manuales difíciles de mantener
- ✗ Muchas pruebas documentadas con poca trazabilidad

### ***Visibilidad y Gestión***

- ✗ Baja visibilidad sobre cobertura real
- ✗ Baja visibilidad sobre estado de pipelines
- ✗ Baja visibilidad sobre métricas de calidad
- ✗ Falta de claridad sobre ownership de calidad
- ✗ Falta de estrategia transversal de QA

### ***Organización***

- ✗ Testers manuales no técnicos (si existen)
- ✗ Falta de liderazgo de QA en algunos equipos
- ✗ Incidentes en producción frecuentes
- ✗ Resistencia moderada en algunos equipos

## **Mi Principio Fundamental como QA Lead**

Cuando ingreso a un proyecto nuevo, mi primer paso es realizar un:

DIAGNÓSTICO DEL ESTADO ACTUAL DE QA, AUTOMATIZACIÓN  
Y COBERTURA

No mido únicamente cuántas pruebas existen.

Determino: ¿Las pruebas realmente protegen las funcionalidades críticas del negocio?

## **Enfoque de Diagnóstico (4 Dimensiones)**

### ***1. DIMENSIÓN TÉCNICA***

Análisis de automatización existente:

- Pruebas funcionales y no funcionales
- Pruebas unitarias (cobertura real vs documentada)
- Pruebas de integración/API
- Pruebas E2E
- Regresiones automatizadas
- CI/CD: ¿Qué se ejecuta? ¿Qué es Quality Gate?
- Estabilidad: Flakiness, pass rate, tiempo de ejecución

Diferenciación de cobertura:

- Cobertura documentada
- Cobertura automatizada
- Cobertura ejecutada
- Cobertura exitosa
- Cobertura confiable
- Cobertura de riesgo ← La más importante

## **2. DIMENSIÓN ORGANIZACIONAL**

Mapeo de subequipos:

- ¿Qué módulo maneja cada subequipo?
- ¿Quién es responsable de QA hoy?
- ¿Cuál es el nivel de madurez en testing?
- ¿Dónde está la resistencia? ¿Por qué?
- ¿Quiénes podrían ser QA Champions?

**Stakeholder mapping:**

- Product Owners (ruta crítica de negocio)
- Tech Leads/Architects (arquitectura)
- DevOps (pipelines y ambientes)
- Sponsors ejecutivos (apoyo organizacional)

**Cultura de calidad:**

- ¿Quién es owner de la calidad hoy?
- ¿Existen ceremonias de calidad?
- ¿Cómo reacciona la organización a incidentes?

### **3. DIMENSIÓN DE PROCESOS Y HERRAMIENTAS**

#### **SDLC actual:**

- Metodología (Scrum, Kanban, SAgFe)
- Sprint length: 1 semana (confirmado)
- Definition of Done: ¿Incluye criterios de calidad?
- ¿Cuándo interviene QA en el ciclo?

#### **Ambientes:**

- ¿Cuántos ambientes? (Dev, QA, Staging, Prod)
- ¿Estabilidad?
- ¿Gestión?

#### **Datos de prueba:**

- ¿Cómo se crean/gestionan?
- ¿Existen datasets consistentes?
- ¿Estrategia de test data management?

#### **Herramientas actuales:**

- Test management (Jira, TestRail, Zephyr, etc.)
- CI/CD (Jenkins, GitLab, GitHub Actions)
- Defect tracking
- Repositorios

### **4. DIMENSIÓN DE MÉTRICAS BASELINE**

#### **Métricas de calidad actuales (últimos 3-6 meses):**

- Incidentes en producción (cantidad, severidad, frecuencia)
- Defect leakage rate (% defectos en prod vs pre-prod)
- Regression rate (frecuencia de regresiones post-deploy)
- Time to detection
- Mean time to resolution (MTTR)

#### **Métricas de delivery:**

- Deployment frequency
- Deployment success rate

- Lead time (commit → producción)
- Rollback rate

#### **Métricas de automatización:**

- Cobertura documentada vs real
- Pass rate de pruebas automatizadas
- Flakiness rate
- Tiempo de ejecución de suites

#### **Identificación de Ruta Crítica**

Si la ruta crítica no existe, la construyo mediante workshops con stakeholders.

#### **Criterios de priorización:**

| Criterio                     | Peso | Método                                     |
|------------------------------|------|--------------------------------------------|
| <i>Impacto en usuarios</i>   | 25%  | Cantidad de usuarios afectados si falla    |
| <i>Impacto en negocio</i>    | 25%  | Revenue, SLA, reputación                   |
| <i>Frecuencia de uso</i>     | 15%  | Analytics, logs                            |
| <i>Historial de defectos</i> | 15%  | Incidentes en producción (últimos 6 meses) |
| <i>Frecuencia de cambios</i> | 10%  | Git history, commits                       |
| <i>Complejidad técnica</i>   | 10%  | Análisis de código, dependencias           |

#### **Salida esperada:**

- Top 10-15 flujos críticos priorizados
- Matriz de riesgo vs cobertura actual

#### **Objetivo de Automatización**

##### **La estrategia busca aproximadamente:**

95% de cobertura de las funcionalidades críticas y de mayor riesgo

NO es: “95% de todas las pruebas automatizadas”

ES: 95% de cobertura de funcionalidades críticas priorizadas por riesgo.

Las regresiones frecuentes buscan automatizarse al:

100% cuando técnicamente sea viable

---

ESTRATEGIA DE QA (1.1)

Visión

Transformar ACME Corp de una organización con QA reactivo a una cultura de Quality Engineering donde la calidad es responsabilidad compartida, apoyada por automatización inteligente, IA como multiplicador de fuerza y visibilidad continua.

Principios Rectores

- 1. Calidad como responsabilidad compartida (no solo QA)
- 2. Priorización basada en riesgo e impacto al negocio
- 3. Automatización estratégica (no automatizar por automatizar)
- 4. IA como multiplicador de fuerza (crítico con equipo de 2 personas)
- 5. Visibilidad continua (métricas, dashboards, trazabilidad)
- 6. Shift-left (calidad desde diseño, no solo en testing)
- 7. Mejora continua (learn, measure, adapt)

Los 5 Pilares de la Estrategia

PILAR 1: OWNERSHIP Y MODELO ORGANIZACIONAL

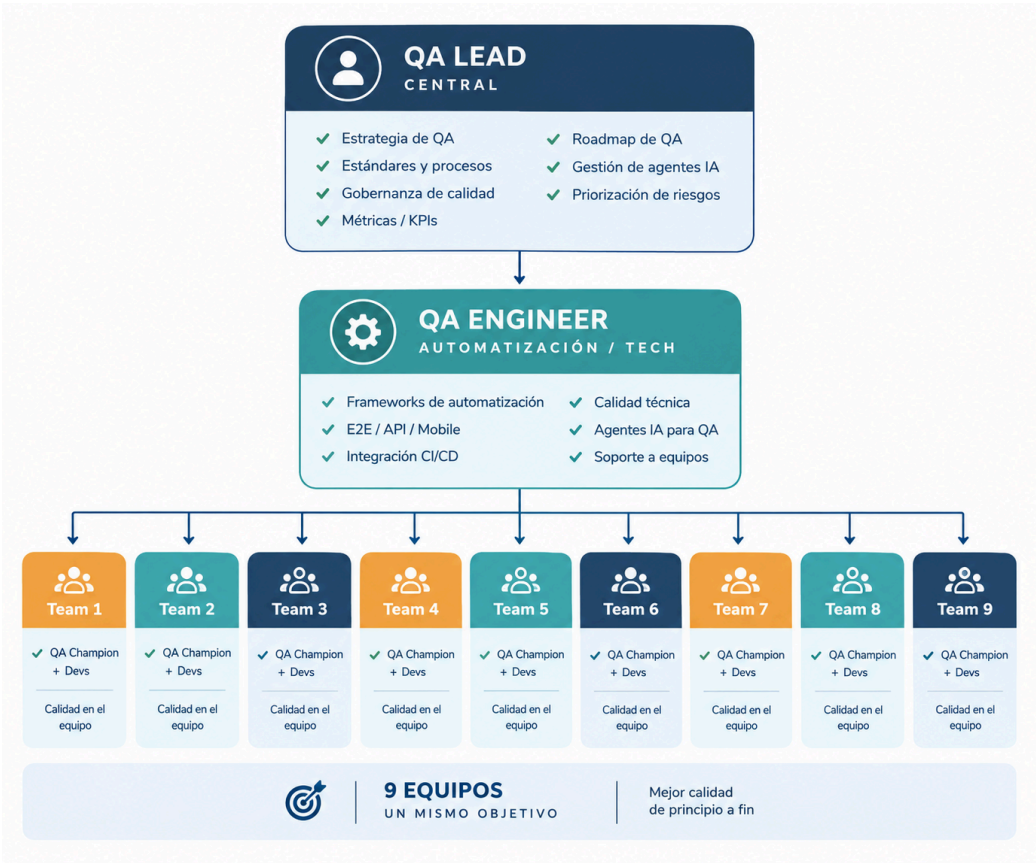
Problema: Falta claridad sobre quién es responsable de calidad. 2 personas de QA no pueden ser gatekeepers de 9 equipos.

Solución: Calidad como responsabilidad compartida

| Rol                            | Responsabilidades en Calidad                                                                        |
|--------------------------------|-----------------------------------------------------------------------------------------------------|
| QA Lead                        | Estrategia de calidad, estándares, Quality Gates, métricas organizacionales, facilitación, training |
| QA Engineer                    | Automatización crítica, frameworks, agentes de IA, soporte a equipos, code reviews de tests         |
| Developers                     | Pruebas unitarias (ownership 100%), pruebas de integración, fix de bugs, code quality               |
| QA Champions (1 por subequipo) | Evangelizar calidad en su equipo, ejecutar estrategia localmente, primeros usuarios de agentes IA   |

| Rol            | Responsabilidades en Calidad                                                                   |
|----------------|------------------------------------------------------------------------------------------------|
| Product Owners | Criterios de aceptación claros, priorización de bugs críticos, validación de flujos de negocio |
| Tech Leads     | Arquitectura testeable, code reviews, balance técnico vs velocidad                             |
| DevOps         | Ambientes estables, pipelines, Quality Gates, instrumentación                                  |

Modelo operativo: QA Central + QA Champions

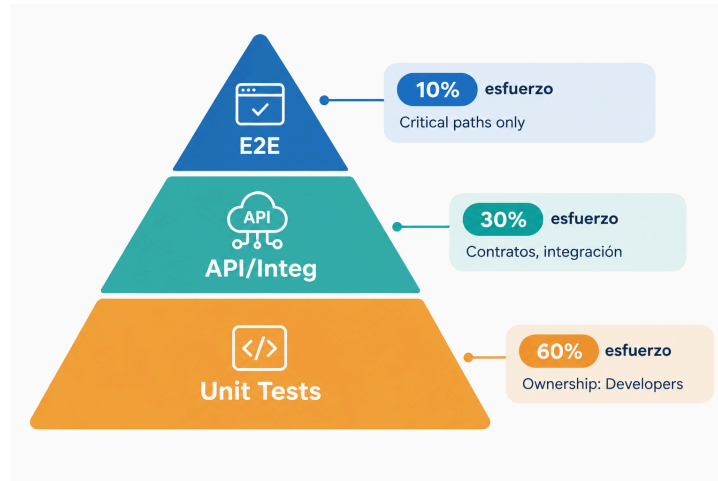


Cómo escala:

- QA Central define estándares, crea agentes, genera frameworks
- QA Champions ejecutan localmente en sus equipos
- Developers son owners de pruebas unitarias y calidad de código
- QA Central no es cuello de botella, es facilitador/multiplicador

## *PILAR 2: TESTING STRATEGY & PYRAMID*

Testing Pyramid adaptado a restricciones:



Justificación:

- Unitarias (60%): Rápidas, confiables, bajo costo de mantenimiento. Developers deben ser owners.
- Integración/API (30%): Balance entre cobertura y velocidad. Crítico para plataforma de mensajería.
- E2E (10%): Solo flujos críticos de usuario. Alto costo, flakiness, mantenimiento. Usar con criterio.

Priorización: Ruta Crítica + Regresiones Frecuentes

| Tipo de funcionalidad                   | Target de cobertura               | Estrategia                                                        |
|-----------------------------------------|-----------------------------------|-------------------------------------------------------------------|
| Funcionalidades críticas (ruta crítica) | 95%                               | Automatización prioritaria: unitarias + integración + E2E crítico |
| Regresiones frecuentes                  | 100% (cuando técnicamente viable) | Automatizar inmediatamente, integrar a CI/CD como Quality Gate    |
| Funcionalidades de riesgo medio         | 70%                               | Automatizar gradualmente                                          |
| Funcionalidades de bajo riesgo          | 40-50%                            | Pruebas manuales exploratorias, automatizar si hay tiempo         |

Estrategia de automatización por capa:

| Capa            | Framework                         | Responsable                | Integración CI/CD | Target coverage                  |
|-----------------|-----------------------------------|----------------------------|-------------------|----------------------------------|
| Unit            | Jest / JUnit / PyTest             | Developers                 | Cada commit       | 80% code coverage                |
| API/Integración | Supertest / RestAssured / Postman | QA Engineer + Developers   | Cada PR           | 95% endpoints críticos           |
| E2E             | Playwright / Cypress              | QA Engineer + QA Champions | Pre-deploy        | 100% flujos críticos (Top 10-15) |
| Performance     | k6 / JMeter                       | QA Engineer + DevOps       | Nightly / Weekly  | Baseline + smoke                 |
| Security        | SAST/DAST                         | DevOps + QA Lead           | Cada release      | OWASP Top 10                     |

Manejo de pruebas manuales:

Cuándo son necesarias:

- Pruebas exploratorias (creatividad, UX, edge cases no documentados)
- Pruebas de usabilidad
- Validación de diseño/UI en diferentes dispositivos
- Flujos complejos aún no automatizados (temporal)
- Smoke testing de funcionalidades nuevas antes de automatizar

Estándares mínimos:

- Toda prueba manual debe tener trazabilidad (requisito → caso de prueba)
- Casos documentados en herramienta central (Jira/Xray/TestRail)
- Si una prueba manual se ejecuta >3 veces, evaluar automatización
- Priorizar automatización según frecuencia de ejecución

---

### ***PILAR 3: IA COMO MULTIPLICADOR DE FUERZA***

Contexto crítico: Solo 2 personas de QA para 9 subequipos = IA es FUNDAMENTAL para escalar.

Los 4 Agentes Especializados:

#### ***AGENTE 1: Analizador de Cobertura Automatizada***

Objetivo: Identificar gaps de cobertura en pruebas existentes.

Capacidades:

- Analizar repositorios de código y pruebas
- Mapear funcionalidades vs pruebas automatizadas
- Detectar funcionalidades sin cobertura
- Identificar pruebas duplicadas, obsoletas, no ejecutadas
- Calcular cobertura real (no solo documentada)
- Generar matriz de cobertura por subequipo

Tecnología: LLM (GPT-4/Claude) + análisis estático + SonarQube/CodeCov

ROI: Reduce análisis manual de semanas → horas

---

## **AGENTE 2: Analizador de Código (Clean Code + Architecture)**

Objetivo: Garantizar que código nuevo y pruebas sigan estándares de calidad.

Principios configurados:

Clean Code:

- SOLID, DRY, KISS, YAGNI
- Funciones <30 líneas (soft limit)
- Nombres descriptivos
- Evitar if/else anidados profundos
- No código muerto, comentarios obsoletos
- Composición sobre herencia

Arquitectura:

- Respetar estructura existente antes de crear
- No crear carpetas/archivos si ya existe equivalente
- Seguir patrones del proyecto

Tipado fuerte (TypeScript):

- strict: true
- Evitar any
- Tipos de retorno explícitos
- No @ts-ignore sin justificación

Regla fundamental: NO INVENTAR

“Antes de implementar, inspecciona código existente, dependencias, configuración. No inventes endpoints, clases, métodos que no existan.”

Quality Gates automáticos:

1. Formatter (Prettier / Black)

2. Linter (ESLint / Pylint)
3. Type check (tsc)
4. Unit tests (pass + coverage threshold)
5. Integration/E2E tests
6. Security scan (SAST)

Code Review automático: El agente hace code review como Senior Engineer antes de aprobar PR.

ROI: Reduce tiempo de code review + mejora calidad consistente

---

### **AGENTE 3: Diseñador de Casos de Uso (Ver documento 2 para detalle completo)**

Objetivo: Generar casos de prueba de alta calidad con trazabilidad completa.

Capacidades:

- Análisis de requisitos (reglas de negocio, gaps, ambigüedades)
- Generación de casos aplicando técnicas ISTQB (BVA, equivalence partitioning, decision tables)
- Matriz de trazabilidad (requisito → prueba)
- Detección de gaps de cobertura
- Exportación a Jira/Xray, CSV, Markdown
- Código de automatización (borrador)

ROI: 75% ahorro de tiempo en diseño de casos de prueba (de 3 horas → 45 min por HU)

Stack: Node.js + AWS Lambda + Serverless + OpenAI/Claude + DynamoDB

---

### **AGENTE 4: Monitor de Métricas de QA**

Objetivo: Generar visibilidad continua sobre calidad mediante dashboards y alertas automáticas.

Métricas rastreadas:

CALIDAD:

- Defect leakage rate
- Production incidents
- Regression rate
- Defect escape rate
- Severity distribution

#### AUTOMATIZACIÓN:

- Critical flow coverage (%)
- Automation coverage
- Test pass rate (%)
- Flakiness rate (%)
- Execution time (trends)

#### CI/CD:

- Pipeline failure rate (%)
- Quality Gate pass rate (%)
- Mean time to feedback
- Deployment confidence score

#### NEGOCIO:

- Flujos críticos protegidos
- Incidentes que afectan clientes
- Riesgo residual

#### Dashboards generados:

1. Executive Dashboard (semanal): Health score general, incidentes, cobertura crítica, deployment success
2. QA Dashboard (diario): Cobertura por equipo, pass rate, flaky tests, gap analysis
3. Team Dashboard (por subequipo): Cobertura local, defectos, Quality Gates, tendencias

### Alertas automáticas:

-  Critical test failed
-  Deployment bloqueado por Quality Gate
-  Flakiness >10% detectado
-  Cobertura de riesgo crítico <95%
-  Incremento de defect leakage

Tecnología: Grafana/Kibana + Prometheus + Python + integración Jira/CI/CD

### ROI: Visibilidad 24/7 sin intervención humana

---

Priorización de implementación de agentes (con 2 personas, 90 días):

| Fase       | Agente                                     | Justificación                              | Esfuerzo    |
|------------|--------------------------------------------|--------------------------------------------|-------------|
| 0-30 días  | Agente 1 (Cobertura) + Agente 4 (Métricas) | Diagnóstico rápido + visibilidad inmediata | 2-3 semanas |
| 31-60 días | Agente 2 (Clean Code)                      | Integrar en CI/CD para calidad de código   | 2 semanas   |
| 61-90 días | Agente 3 (Casos de uso) - MVP              | Escalar diseño de pruebas                  | 3-4 semanas |

---

### **PILAR 4: CI/CD & QUALITY GATES**

Integración de calidad en pipeline:



### Quality Gates definidos:

| Stage      | Quality Gate       | Criterio                         | Acción si falla                    |
|------------|--------------------|----------------------------------|------------------------------------|
| PR         | Unit tests         | Pass rate 100%                   | Bloquear merge                     |
| PR         | Code coverage      | Delta coverage $\geq 0\%$        | Warning (no bloquear inicialmente) |
| PR         | Integration tests  | Pass rate 100%                   | Bloquear merge                     |
| PR         | Security scan      | No critical/high vulnerabilities | Bloquear merge                     |
| Pre-deploy | E2E critical paths | Pass rate 100%                   | Bloquear deploy                    |
| Pre-deploy | Regression suite   | Pass rate $\geq 95\%$            | Manual approval requerido          |
| Pre-deploy | Performance smoke  | Response time < threshold        | Manual approval requerido          |

### Configuración gradual:

- Primeros 30 días: Quality Gates en warning mode (no bloquean)
- 30-60 días: Habilitar bloqueo en PR (unit + integration)
- 60-90 días: Habilitar bloqueo en deploy (E2E + regression)

### Quality Gates progresivos basados en riesgo:

|                       |                                                                                           |                        |
|-----------------------|-------------------------------------------------------------------------------------------|------------------------|
| Tipo de cambio        | Validación requerida                                                                      | Puede bloquearse       |
| Cambio de bajo riesgo | Peer review + pruebas automatizadas + smoke                                               | No                     |
| Cambio de alto riesgo | Pruebas funcionales específicas + regresión flujo crítico + evidencia                     | Sí (si falla critical) |
| Cambio crítico (Po)   | Pruebas automatizadas 100% + integración + validación negocio + monitoreo + rollback plan | Sí                     |

## PILAR 5: VISIBILIDAD Y MÉTRICAS ORGANIZACIONALES

KPIs por audiencia:

Para Dirección / Ejecutivos (mensual):

- Production incidents trend
- Deployment success rate
- Critical flows coverage
- Time to market (impacto de calidad)
- Customer-impacting defects

Para Engineering Managers (semanal):

- Team quality score
- Defect leakage rate
- Automation coverage
- Quality Gate pass rate
- Technical debt de testing

Para QA y Developers (diario):

- Test pass rate
- Flaky tests
- Coverage gaps
- Failed builds
- Open defects

Reportes automatizados (generados por Agente 4):

- Daily: Health check a equipos (pass rate, critical failures)
  - Weekly: Executive summary a dirección
  - Sprint: Retrospective de calidad (qué mejoró, qué empeoró)
  - Release: Quality report (cobertura, riesgos, incidentes)
-

Priorización: **NOW / NEXT / LATER**

 **NOW** (0-30 días) — *Quick Wins + Baseline*

| <b>Acción</b>                                  | <b>Responsable</b>      | <b>Impacto</b> | <b>Esfuerzo</b> |
|------------------------------------------------|-------------------------|----------------|-----------------|
| <b>Diagnóstico completo</b>                    | QA Lead                 | Alto           | 2 semanas       |
| <b>Implementar Agente 1 (Cobertura)</b>        | QA Engineer             | Alto           | 2 semanas       |
| <b>Implementar Agente 4 (Métricas) - MVP</b>   | QA Engineer             | Alto           | 1 semana        |
| <b>Identificar ruta crítica</b>                | QA Lead + Product       | Alto           | 1 semana        |
| <b>Identificar QA Champions</b>                | QA Lead + Managers      | Alto           | 1 semana        |
| <b>Automatizar top 3 regresiones críticas</b>  | QA Engineer + Champions | Alto           | 2 semanas       |
| <b>Configurar Quality Gates (warning mode)</b> | QA Engineer + DevOps    | Medio          | 1 semana        |
| <b>Dashboard ejecutivo v1</b>                  | QA Lead                 | Alto           | 1 semana        |

Resultado esperado: Baseline establecido, 6-8 flujos críticos automatizados, visibilidad organizacional, quick wins visibles.

 **NEXT (31-60 días) — Estabilización + Automatización prioritaria**

| Acción                                        | Responsable             | Impacto | Esfuerzo  |
|-----------------------------------------------|-------------------------|---------|-----------|
| Automatizar top 10-15 flujos críticos         | QA Engineer + Champions | Alto    | 3 semanas |
| Implementar Agente 2 (Clean Code)             | QA Engineer             | Alto    | 2 semanas |
| Integrar Quality Gates en PR (bloqueo activo) | QA Engineer + DevOps    | Alto    | 1 semana  |
| Training a QA Champions (automatización)      | QA Lead                 | Medio   | 2 semanas |
| Refactorizar pruebas flaky/duplicadas         | QA Engineer             | Medio   | Continuo  |
| Expandir cobertura integración/API            | QA Engineer + Devs      | Alto    | 3 semanas |

Resultado esperado: 95% de flujos críticos automatizados, Quality Gates activos en PR, Champions capacitados.

 **LATER (61-90 días) — Escalamiento + Quality Engineering**

| Acción                                       | Responsable          | Impacto | Esfuerzo  |
|----------------------------------------------|----------------------|---------|-----------|
| Implementar Agente 3 (Casos de uso)          | QA Engineer          | Medio   | 3 semanas |
| Shift-left: Integrar calidad en refinamiento | QA Lead + POs        | Alto    | Continuo  |
| Developers ownership de unit tests (80%)     | Tech Leads           | Alto    | Continuo  |
| Quality Gates en deploy (bloqueo activo)     | QA Engineer + DevOps | Alto    | 1 semana  |
| Expandir performance testing                 | QA Engineer + DevOps | Medio   | 2 semanas |
| Retrospectivas de calidad organizacionales   | QA Lead              | Medio   | Mensual   |

Resultado esperado: Cultura de Quality Engineering establecida, ownership compartido, trazabilidad completa, pipelines maduros.

---

**Qué Dejo Fuera (Y Por Qué)**

| Qué                                | Por qué lo dejo para después (post-90 días)                             |
|------------------------------------|-------------------------------------------------------------------------|
| Pruebas de carga/stress completas  | Quick wins en funcionalidad. Performance smoke suficiente inicialmente. |
| Automatización de pruebas visuales | Alto esfuerzo, bajo ROI inicial. Priorizar funcionalidad.               |
| Testing cross-browser extensivo    | Enfocarse en browser principal + mobile. Expandir después.              |
| Pruebas de accesibilidad           | Importante pero no bloqueante para MVP. Post-90 días.                   |
| Chaos engineering                  | Requiere mayor madurez. Implementar cuando pipelines estén estables.    |
| Agente 3 completo                  | MVP en 90 días, full implementation después.                            |

## IMPLEMENTACIÓN CON RESTRICCIONES (1.2)

### Restricciones Reales

- 🕒 90 días para demostrar impacto
- 👥 Solo 2 personas en QA
- 🚧 Resistencia moderada en algunos equipos
- 🏢 9 subequipos con diferentes niveles de madurez
- 📈 Sprints de 1 semana (alta velocidad)
- 🚀 Deploys frecuentes

### Plan de Ejecución: Primeros 14 Días

Objetivo: Entender el ecosistema, identificar flujos críticos, establecer estándar mínimo de calidad, comenzar automatización de alto valor.

### Distribución de Responsabilidades

| Actividad                      | QA Lead          | QA Engineer  | IA                          |
|--------------------------------|------------------|--------------|-----------------------------|
| <b>Estrategia QA</b>           | ✅ Responsable    | 👉 Apoya      | 🤖 Analiza/propone           |
| <b>Priorización por riesgo</b> | ✅ Responsable    | 👉 Apoya      | 🤖 Identifica riesgos        |
| <b>Entrevistas 9 equipos</b>   | ✅ Lidera         | 👉 Participa  | 🤖 Resume y consolida        |
| <b>Análisis arquitectura</b>   | ✅ Lidera         | ✅ Ejecuta    | 🤖 Analiza docs/código       |
| <b>Casos de prueba</b>         | 👉 Supervisa      | ✅ Ejecuta    | 🤖 Genera borradores         |
| <b>Automatización</b>          | 👉 Define roadmap | ✅ Implementa | 🤖 Genera/refactoriza código |
| <b>CI/CD</b>                   | 👉 Define gates   | ✅ Implementa | 🤖 Analiza fallos            |
| <b>Métricas QA</b>             | ✅ Define         | 👉 Alimenta   | 🤖 Consolida                 |
| <b>Reporte ejecutivo</b>       | ✅ Responsable    | 👉 Apoya      | 🤖 Genera borrador           |

---

 DÍAS 1-2: Discovery y Diagnóstico

Objetivo: ¿Qué tenemos? ¿Qué tan crítico es? ¿Dónde están los mayores riesgos?

*QA Lead (Día 1)*

Mañana: - Reunión inicial con sponsor/manager (1h): Alinear expectativas, solicitar accesos - Revisión de documentación existente (2h): Arquitectura, roadmap, org chart

Tarde: - Análisis de incidentes históricos (3h): Últimos 3-6 meses, severidad, módulos afectados - Preparar matriz de entrevistas (1h): Agendar 9 Tech Leads + POs + DevOps

*QA Engineer (Día 1)*

Assessment técnico: - Mapear repositorios (cantidad, lenguajes, estructura) - Analizar tests existentes (unitarios, integración, E2E) - Analizar CI/CD (pipelines, ejecución automática, pass rate) - Analizar ambientes (cuántos, estabilidad, datos)

Output: Documento “Estado actual de automatización y CI/CD”

*IA (Día 1-2)*

Análisis automáticos: - Agente 1: Analizar repos, identificar tests, detectar duplicación, módulos sin pruebas - Análisis de HUs en Jira: Frecuencia de defectos, criterios de aceptación

Output: Reporte de gaps de cobertura por módulo

*Sincronización (Final día 2)*

QA Lead + QA Engineer consolidan hallazgos técnicos + organizacionales.

---

 DÍAS 3-5: Entrevistas con los 9 Equipos

Objetivo: Identificar ruta crítica, madurez, resistencia, QA Champions potenciales.

Formato: 45 min por equipo con Tech Lead + PO

### *Preguntas Clave*

| Pregunta                                | Objetivo                      |
|-----------------------------------------|-------------------------------|
| ¿Qué construyen?                        | Entender dominio              |
| ¿Qué es crítico?                        | Identificar ruta crítica      |
| ¿Qué puede generar pérdida de dinero?   | Priorizar por impacto negocio |
| ¿Qué puede afectar al cliente?          | Priorizar por impacto usuario |
| ¿Qué integraciones tienen?              | Identificar dependencias      |
| ¿Qué se rompe frecuentemente?           | Identificar regresiones       |
| ¿Qué pruebas existen?                   | Baseline de cobertura         |
| ¿Qué está automatizado?                 | Gaps de automatización        |
| ¿Quién tiene interés/skills en calidad? | Identificar QA Champions      |
| ¿Percepción sobre QA?                   | Detectar resistencia          |

### *Distribución*

| Día          | QA Lead                                    | QA Engineer                    |
|--------------|--------------------------------------------|--------------------------------|
| <b>Día 3</b> | Entrevistar equipos 1, 2, 3 + PO principal | Participar + tomar notas       |
| <b>Día 4</b> | Entrevistar equipos 4, 5, 6 + DevOps       | Participar + consolidar matriz |
| <b>Día 5</b> | Entrevistar equipos 7, 8, 9                | Participar + preparar síntesis |

Output: Matriz de Equipos (Ejemplo)

| Equipo | Dominio       | Criticidad | Automatizaci3n | Madurez | Riesgo  | Resistencia | QA Champion | Prioridad |
|--------|---------------|------------|----------------|---------|---------|-------------|-------------|-----------|
| Team 1 | Auth          | Alta       | 20%            | Baja    | Alto    | Media       | Juan P.     | Po        |
| Team 2 | Messaging     | Alta       | 35%            | Media   | Alto    | Baja        | María G.    | Po        |
| Team 3 | Payments      | Alta       | 15%            | Baja    | Crítico | Alta        | —           | Po        |
| Team 4 | Notifications | Media      | 50%            | Alta    | Medio   | Baja        | Carlos R.   | P1        |

Gesti3n de Resistencia:

Señales: “Ya hacemos suficientes pruebas”, “QA ralentiza”, “No hay tiempo”

Estrategia: NO confrontar, escuchar pain points, mostrar datos > opiniones, buscar aliados internos

 DÍAS 6-7: Mapa de Riesgos y Ruta Crítica

Decisi3n fundamental: NO podemos cubrir 9 equipos uniformemente con 2 personas.

Clasificaci3n de Criticidad

| Nivel              | Criterio                                                          | Acciones                                          |
|--------------------|-------------------------------------------------------------------|---------------------------------------------------|
| Po — Critical Path | Pérdida econ3mica, afecta pagos, seguridad, datos, disponibilidad | QA activo, automatizaci3n prioritaria             |
| P1 — Alta          | Importante, impacto controlable                                   | Automatizaci3n gradual                            |
| P2 — Media/Baja    | Menor impacto                                                     | Pruebas exploratorias, automatizaci3n oportunista |

Ejemplo de clasificación: - Po: Teams 1 (Auth), 2 (Messaging), 3 (Payments), 9 (Integrations) → 4 equipos - P1: Teams 4, 5, 8 → 3 equipos - P2: Teams 6, 7 → 2 equipos

#### *Construcción de Ruta Crítica con Product (Día 6 tarde)*

Pregunta clave: “¿Cuáles son los 10-15 flujos que NUNCA pueden fallar?”

Matriz de priorización:

| Flujo crítico   | Equipo | Impacto negocio | Impacto usuario | Frecuencia | Historial defectos | Score | Prioridad |
|-----------------|--------|-----------------|-----------------|------------|--------------------|-------|-----------|
| Login/Auth      | Team 1 | 10              | 10              | 10         | 8                  | 45    | 1         |
| Send message    | Team 2 | 10              | 10              | 10         | 9                  | 47    | 1         |
| Process payment | Team 3 | 10              | 10              | 8          | 7                  | 40    | 2         |

Output: Top 10-15 flujos críticos priorizados

#### *Preparación de Comunicación (Día 7)*

Presentación preliminar a dirección/managers (día 8): 1. Hallazgos de discovery 2. Mapa de riesgos (Po/P1/P2) 3. Ruta crítica 4. Gaps críticos 5. Plan de acción 6. Expectativas realistas: Qué SÍ, qué NO haremos aún

---

#### DÍAS 8-10: Quality Baseline

#### *Definition of Ready (DoR)*

Una HU NO entra a desarrollo si: - No tiene criterios de aceptación claros - No se entiende comportamiento esperado - Dependencias desconocidas - Faltan reglas de negocio críticas - No se identificaron riesgos

#### *Definition of Done (DoD)*

Antes de producción: - Código revisado (peer review) - Tests unitarios (según corresponda) - Pruebas funcionales ejecutadas - Escenarios críticos validados - Defectos críticos resueltos - Evidencia disponible - Riesgos documentados

Calidad es responsabilidad compartida:

| Rol         | Responsabilidad                                                |
|-------------|----------------------------------------------------------------|
| Product     | Criterios de aceptación claros, priorización bugs              |
| Development | Pruebas unitarias, code quality, fix bugs                      |
| QA          | Estrategia pruebas, automatización, validación flujos críticos |
| DevOps      | Ambientes estables, pipelines, Quality Gates                   |

Quality Gates Basados en Riesgo

| Tipo de cambio | Validación                                                         | Responsable        | Bloqueo         |
|----------------|--------------------------------------------------------------------|--------------------|-----------------|
| Bajo riesgo    | Peer review + automatizadas + smoke                                | Dev + CI/CD        | No              |
| Alto riesgo    | Funcionales + regresión + evidencia                                | QA + Dev           | Sí (si crítico) |
| Crítico (Po)   | Automatizadas 100% + integración + negocio + monitoring + rollback | QA + Dev + Product | Sí              |

Comunicación del Baseline (Día 10): Compartir DoR/DoD con 9 equipos, solicitar feedback.

 DÍAS 11-12: Primer Quick Win de Automatización

NO intentar automatizar 100 casos. Seleccionar 3-5 flujos de MAYOR riesgo.

Prioridad de capas: 1. API/Integration (más rápido, más estable) 2. E2E crítico (solo happy path) 3. Integraciones (contratos) 4. UI secundaria

Ejemplo de Quick Win para ACME Corp:

| Flujo crítico        | Tipo        | Esfuerzo | Impacto              |
|----------------------|-------------|----------|----------------------|
| Login/Auth           | API + E2E   | 1 día    | Protege acceso total |
| Send message         | API + E2E   | 1 día    | Core del negocio     |
| Process payment      | API         | 0.5 día  | Revenue crítico      |
| Receive notification | Integration | 0.5 día  | Engagement           |

Total: 3 días → 4 flujos críticos automatizados en CI/CD

QA Engineer: Con ayuda IA, genera código, revisa, ejecuta, integra en pipeline (warning mode)

QA Lead: Mientras tanto, identificar formalmente QA Champions (reuniones 1:1)

---

 **DÍAS 13-14: Dashboard y Métricas**

Implementar Agente 4 (Métricas) — MV

Dashboard v1 consolidando: - Jira (defectos, HUs) - Repos (cobertura, tests) - CI/CD (pass rate, tiempo) - Monitoring (incidente)





## Dashboard Inicial:



### Presentación Día 14

Estructura (1h con stakeholders): 1. Lo que descubrimos (5 min) 2. Lo que hicimos (5 min) 3. Lo que haremos (10 min) 4. Expectativas realistas (5 min) 5. Q&A (5 min)

Objetivo: Transparencia total sobre lo hecho, lo que se hará, lo que NO se hará.

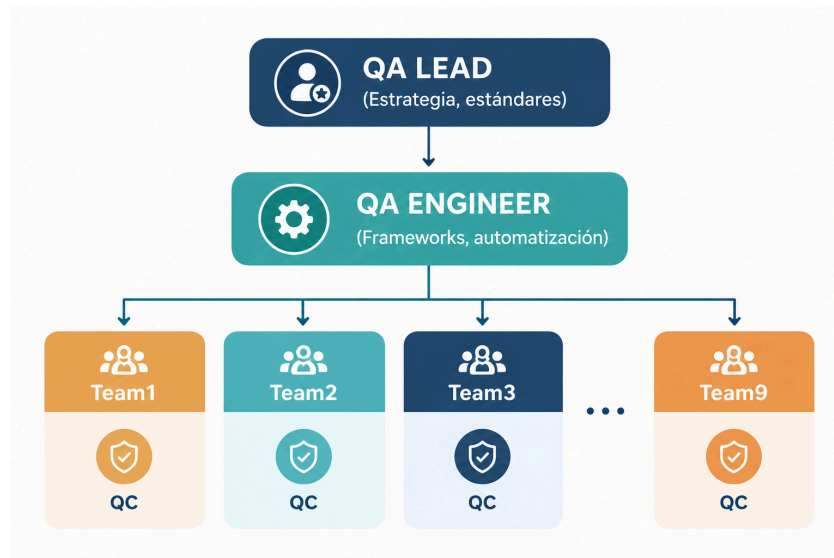
---

### Métricas de Éxito (Primeros 14 Días)

| Métrica                 | Target            | ¿Cumplido? |
|-------------------------|-------------------|------------|
| Equipos evaluados       | 9/9               | ✓          |
| Ruta crítica definida   | Top 10-15 flujos  | ✓          |
| Baseline establecido    | DoR + DoD + Gates | ✓          |
| Quick wins              | 3-5 flujos        | ✓          |
| Integración CI/CD       | Sí                | ✓          |
| Champions identificados | 4-9               | ✓          |
| Dashboard v1            | Sí                | ✓          |
| Presentaciones          | 2 (día 8 + 14)    | ✓          |

### Qué Delego y Cómo

Con 2 personas, delego calidad dentro de equipos mediante QA Champions.



Rol de QA Champion: - Definición de escenarios - Ejecución de pruebas exploratorias - Validar HUs (criterios claros) - Promover pruebas unitarias - Identificar riesgos temprano - Mantener automatización local

Beneficio: QA Central es facilitador, no isla que prueba todo.

### Qué NO Hago (Días 1-14)

Conscientemente:

- Automatizar todos los módulos
- Crear framework desde cero sin evaluar
- Crear cientos de casos sin priorizar
- Cubrir 9 equipos uniformemente
- Convertir QA en cuello de botella
- Bloquear releases por bugs de bajo riesgo
- Medir éxito solo por cantidad de casos
- Usar IA sin validación humana
- Reemplazar unitarias por E2E
- Perseguir 100% code coverage inicial
- Reuniones con 60 developers
- Procesos burocráticos

### Plan de Contingencia

| Bloqueador              | Acción                                          |
|-------------------------|-------------------------------------------------|
| Tech Lead no disponible | Hablar con dev senior + DevOps                  |
| Sin documentación       | Análisis IA + entrevistas                       |
| Sin CI/CD               | Implementar básico PO, ejecutar manual mientras |
| Repos caóticos          | Enfoque en PO, registrar deuda                  |
| Resistencia PO          | Escalar a manager, datos de incidentes          |
| Agentes IA fallan       | Análisis manual temporal                        |
| No lograr quick wins    | Comunicar, ajustar expectativas                 |

---

ESCALABILIDAD ORGANIZACIONAL (1.4)

Análisis de Escalabilidad

Contexto Actual (60 developers, 9 equipos)

|                  |                                   |
|------------------|-----------------------------------|
| Parámetro        | Valor                             |
| Developers       | 60                                |
| Subequipos       | 9                                 |
| QA team          | 2 personas                        |
| QA Champions     | 9 (1 por equipo)                  |
| Ratio QA:Dev     | 1:30                              |
| HUs por sprint   | ~30-40                            |
| Modelo operativo | QA Central (2) + QA Champions (9) |

Sostenibilidad:  VIABLE con estrategia aprobada

ESCENARIO A: 100 DEVELOPERS (15 EQUIPOS)

Análisis de Impacto

| Elemento          | 60 devs       | 100 devs      | Sostenible?  | Acción                              |
|-------------------|---------------|---------------|--------------|-------------------------------------|
| QA Central        | 2 personas    | 2 personas    | LÍMITE       | Considerar +1 QA                    |
| QA Champions      | 9 equipos     | 15 equipos    | SOBRECARGADO | 2 QAs no gestionan 15 efectivamente |
| Ratio QA:Dev      | 1:30          | 1:50          | ALTO         | Ideal 1:20-30                       |
| HUs por sprint    | 30-40         | 50-65         | ALTO         | QA Copilot 60%+ volumen             |
| Quality Gates     | Automatizados | Automatizados | ESCALA       | No requiere cambio                  |
| Agentes IA        | 4 agentes     | 4 agentes     | ESCALA       | Suficientes                         |
| Discovery inicial | 14 días       | 21-28 días    | MÁS LENTO    | Más tiempo o +1 QA                  |

Puntos de Quiebre Identificados

 CUELLO DE BOTELLA 1: Gestión de QA Champions

Problema: - 2 QAs gestionando 15 Champions - Tiempo por Champion: 2-3 horas/semana - Total: 30-45 horas/semana para gestión - Con 2 personas = 80 horas disponibles - Consumo: 37-56% del tiempo solo en gestión

Estado: ⚠ INSOSTENIBLE (no hay tiempo para trabajo técnico)

#### 🟡 CUELLO DE BOTELLA 2: Volumen de HUs

- 50-65 HUs por sprint
- Con QA Copilot (75% ahorro), revisión:  $65 \times 45 \text{ min} = 49 \text{ horas/sprint}$
- Si Champions revisan 60% = ~20 horas QA Central
- Estado: ⚠ AJUSTADO

✅ NO SON CUELLO DE BOTELLA (Escalan Bien): - Quality Gates automáticos - CI/CD pipelines - Agentes de IA - Automatización de pruebas

---

### SOLUCIÓN PARA 100 DEVELOPERS

OPCIÓN A: Optimizar sin contratar (temporal, 6 meses max)

| Acción                      | Impacto                                                                         |
|-----------------------------|---------------------------------------------------------------------------------|
| Jerarquizar Champions       | 3 Senior Champions gestionan 4-5 juniors cada uno. QA Central → 3 seniors solo. |
| Automatizar training        | Videos, docs, QA Copilot como tutor. 8h → 2h por Champion.                      |
| Delegar más a IA            | QA Copilot procesa 80% HUs (vs 60%). Solo críticas/complejas a humano.          |
| Quality Gates más estrictos | Aumentar nivel para reducir intervención manual.                                |

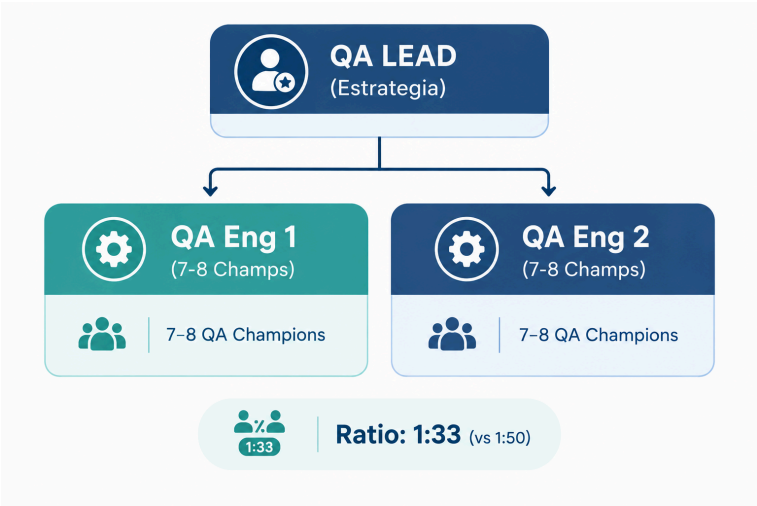
Resultado: Viable 6 meses, QA Central al 90% capacidad (insostenible largo plazo)

---

OPCIÓN B: Contratar +1 QA Engineer (recomendado)

Nuevo equipo: 3 personas - 1 QA Lead (estrategia, gestión Champions) - 2 QA Engineers (automatización, soporte, agentes IA)

Distribución:



Resultado: **Sostenible** largo plazo, 70% capacidad (margen para crecimiento)

*Estrategia Adaptada (100 devs)*

LO QUE SE MANTIENE: - 5 pilares de estrategia - 4 agentes IA - Quality Gates automáticos - Testing pyramid 60/30/10 - Priorización Po/P1/P2 - Calidad compartida

LO QUE CAMBIA:

| Elemento            | 60 devs                   | 100 devs                           |
|---------------------|---------------------------|------------------------------------|
| Equipo QA           | 2 personas                | 3 personas (+1)                    |
| Modelo Champions    | Plano                     | Jerárquico (3 Senior → 12 juniors) |
| Ratio QA:Dev        | 1:30                      | 1:33                               |
| Autonomía Champions | Media (60% HUs)           | Alta (80% HUs)                     |
| Training            | Manual (8h)               | Automatizado (2h)                  |
| Quality Gates       | Warning → bloqueo gradual | Bloqueo estricto desde inicio      |
| Discovery           | 14 días                   | 21 días o paralelo (14 días)       |

ESCENARIO B: 150 DEVELOPERS (20-25 EQUIPOS)

Análisis de Impacto

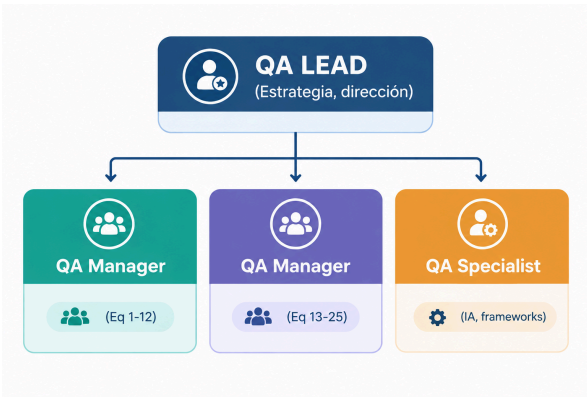
| Elemento       | 100 devs      | 150 devs      | Sostenible?    | Acción                 |
|----------------|---------------|---------------|----------------|------------------------|
| QA Central     | 3 personas    | 3 personas    | ● INSUFICIENTE | +2 QA (total 5)        |
| QA Champions   | 15            | 20-25         | ● SOBRECARGADO | Jerárquico obligatorio |
| Ratio QA:Dev   | 1:33          | 1:50          | ● MUY ALTO     | Contratar obligatorio  |
| HUs por sprint | 50-65         | 80-100        | ● MUY ALTO     | IA 90%+                |
| Quality Gates  | Automatizados | Automatizados | ✓ ESCALA       | OK                     |
| Agentes IA     | 4 agentes     | 4-5 agentes   | ⚠ OPTIMIZAR    | Considerar Agente 5    |

Cuello de Botella Crítico

Con 3 QAs y 25 equipos: - 25 Champions × 2h/semana = 50 horas/semana  
gestión - 3 QAs × 40h = 120 horas disponibles - 42% tiempo en gestión → ●  
INSOSTENIBLE

SOLUCIÓN: Contratar y Reestructurar

Nuevo equipo QA: 5 personas



Roles:

| Rol         | Cantidad | Responsabilidades                                           |
|-------------|----------|-------------------------------------------------------------|
| QA Lead     | 1        | Estrategia, métricas, dirección, stakeholders               |
| QA Manager  | 2        | Gestión 10-12 equipos c/u, training, coordinación Champions |
| QA Engineer | 2        | Automatización, agentes IA, frameworks, soporte técnico     |

Ratio: 1:30

*Estrategia Adaptada (150 devs)*

LO QUE SE MANTIENE: - 5 pilares - Quality Gates automáticos - IA como multiplicador - Calidad compartida

LO QUE CAMBIA:

| Elemento            | 100 devs   | 150 devs                                  |
|---------------------|------------|-------------------------------------------|
| Equipo QA           | 3 personas | 5 personas                                |
| Estructura          | Plana      | 3 niveles (Lead → Managers → Specialists) |
| Modelo Champions    | 1 nivel    | 2 niveles (Senior Champions → Champions)  |
| Autonomía Champions | 80% HUs    | 90% HUs                                   |
| Agentes IA          | 4 agentes  | 5 agentes (Gestor de Champions)           |
| Quality Gates       | Estricto   | Muy estricto + auto-rollback              |
| Training            | Autoguiado | Certificación formal                      |
| Métricas            | Por equipo | Por cluster (grupos 5 equipos)            |

NUEVO: Agente 5 — Gestor de Champions

Objetivo: Automatizar gestión de QA Champions.

Capacidades: - Asignar HUs a Champions según carga/skills - Detectar Champions con bajo performance - Sugerir training personalizado - Alertar sobre bottlenecks - Reportes automáticos de performance

ROI: Reduce gestión de 2h/semana → 30 min/semana por Champion

---

## ESCALABILIDAD SIN PRESUPUESTO PARA CONTRATAR

Escenario realista: Crece a 100-150 devs pero NO hay presupuesto para QA adicional.

*Estrategia: “Máxima Automatización + IA + Priorización Radical”*

Principio: Si no escalo con personas, escalo con tecnología y delego ownership agresivamente.

---

## CAMBIOS OBLIGATORIOS

### 1. PRIORIZACIÓN RADICAL: Solo Po

Con 2 personas y 15-25 equipos:

|                 |                                    |
|-----------------|------------------------------------|
| Antes (60 devs) | Sin presupuesto (100-150 devs)     |
| Po: 4 equipos   | Po: 3-4 equipos (cobertura activa) |
| P1: 3 equipos   | P1: 0 equipos (self-service)       |
| P2: 2 equipos   | P2: 0 equipos (self-service)       |

Criterio Po más estricto:

Solo califican equipos que cumplan TODOS:

|                      |                             |
|----------------------|-----------------------------|
| Criterio             | Umbral                      |
| Impacto económico    | Pérdida >\$100K si falla    |
| Impacto clientes     | >50% usuarios afectados     |
| Compliance/legal     | Regulación crítica          |
| Historial incidentes | ≥3 críticos últimos 6 meses |
| Frecuencia cambios   | >2 deploys/semana           |

Resultado: De 15-25 equipos, solo 3-4 califican Po.

---

## 2. QUALITY GATES MÁS ESTRINGENTES

Si no valido manualmente, gates deben compensar:

| Gate         | Antes (con QA)     | Sin presupuesto                            |
|--------------|--------------------|--------------------------------------------|
| Unit tests   | 100% pass          | 100% pass + $\geq 80\%$ coverage (bloqueo) |
| Integration  | 100% pass          | 100% pass + $\geq 95\%$ cobertura crítica  |
| E2E          | 100% pass          | 100% pass + 0 flaky tests                  |
| Regression   | $\geq 95\%$        | 100% (bloqueo estricto)                    |
| Security     | No critical/high   | No critical/high/medium                    |
| Performance  | Smoke              | Regression $\leq 5\%$                      |
| Code quality | Linter + formatter | + complexity + duplicación $< 3\%$         |

Efecto: Equipos NO hacen merge/deploy si no cumplen estándares altísimos.

---

## 3. IA COMO REEMPLAZO PARCIAL (NO SOLO ACELERADOR)

| Agente   | Antes                      | Sin presupuesto                           |
|----------|----------------------------|-------------------------------------------|
| Agente 1 | Analiza gaps → QA decide   | Auto-genera tickets → asigna a developers |
| Agente 2 | Revisa código → QA aprueba | Auto-rechaza PRs no conformes             |
| Agente 3 | Genera → QA revisa         | Auto-publica Jira (P1/P2, sin revisión)   |
| Agente 4 | Dashboard → QA analiza     | Auto-alerta + auto-crea tickets           |

NUEVO: Agente 5 (Revisor Automatizado)

Simula code review QA para equipos P1/P2: - Revisa PRs automáticamente -  
Valida pruebas adecuadas - Aprueba/rechaza automáticamente - Comenta con sugerencias

Resultado: P1/P2 reciben “code review de IA” en lugar de QA humano.

---

4. DEVELOPERS COMO QA DE PRIMERA LÍNEA (OBLIGATORIO)

| Antes                   | Sin presupuesto                             |
|-------------------------|---------------------------------------------|
| Unit tests recomendados | OBLIGATORIOS (bloqueo <80% coverage)        |
| Integration opcionales  | OBLIGATORIOS APIs críticas                  |
| QA revisa               | Developer valida con QA Copilot antes PR    |
| Exploratorias por QA    | Developer hace exploratorias (checklist IA) |

Training obligatorio: Programa “Developer as Quality Engineer” (1 día, online)

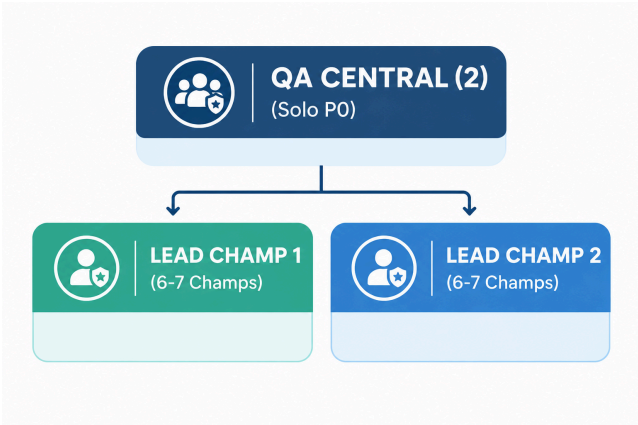
---

5. QA CHAMPIONS: DE “APOYO” A “OWNERS”

| Antes                                    | Sin presupuesto                    |
|------------------------------------------|------------------------------------|
| Champions ejecutan estrategia QA Central | Champions DEFINEN estrategia local |
| QA Central revisa                        | Champions autónomos (QA audita)    |
| Reportan semanalmente                    | Autónomos (solo si alertas)        |
| QA entrena                               | Auto-entrenan (online + IA)        |

Nuevo perfil Champion: - Senior developer (3+ años) - Conocimiento testing - 20-30% tiempo oficialmente asignado a QA

Estructura jerárquica obligatoria (15-25 equipos):



## 6. ELIMINAR TRABAJO DE BAJO VALOR

LO QUE YA NO HACES (P1/P2): - Revisión manual casos prueba → Solo IA - Pruebas exploratorias → Developers + Champions - Reuniones semanales → Solo Po - Code reviews tests → Solo IA - Soporte automatización → Champions (QA solo escala) - Documentación casos → IA + Champions - Bugs no críticos → Developers reportan

EN QUÉ INVIERTES TIEMPO (2 QAs):

| Actividad                     | % Tiempo | Equipos |
|-------------------------------|----------|---------|
| Estrategia y estándares       | 20%      | Todos   |
| Equipos Po (activo)           | 40%      | 3-4     |
| Automatización crítica        | 20%      | Po      |
| Agentes IA (mantenimiento)    | 10%      | Todos   |
| Auditorías trimestrales P1/P2 | 5%       | 11-21   |
| Gestión Lead Champions        | 5%       | 2       |

## 7. AUDITORÍAS EN LUGAR DE COBERTURA CONTINUA

Equipos P1/P2 no tienen cobertura continua, sino auditorías periódicas:

| Frecuencia | Equipos | Alcance      | Responsable         |
|------------|---------|--------------|---------------------|
| Mensual    | Po      | Completa     | QA Central          |
| Trimestral | P1/P2   | Muestreo 20% | QA + Lead Champions |
| Anual      | Todos   | Completa     | QA Central          |

Resultado auditoría:

| Score   | Acción                                                                                                             |
|---------|--------------------------------------------------------------------------------------------------------------------|
| 90-100% |  Mantener autonomía             |
| 75-89%  |  Plan mejora (Champion lidera)  |
| 60-74%  |  Intervención QA (temporal)     |
| <60%    |  Equipo → Po (cobertura activa) |

## COMPARACIÓN: CON vs SIN PRESUPUESTO

| Elemento         | Con presupuesto       | Sin presupuesto           |
|------------------|-----------------------|---------------------------|
| Equipo QA        | Crece (3→5)           | Fijo: 2                   |
| Cobertura activa | 100% equipos          | Solo 3-4 Po               |
| Equipos P1/P2    | Gradual               | Self-service + auditorías |
| Quality Gates    | Estrictos             | MUY estrictos             |
| IA               | Acelerador            | Reemplazo parcial         |
| Developers       | Ownership recomendado | Obligatorio               |
| Champions        | Apoyo                 | Owners autónomos          |
| Lead Champions   | Opcional              | Obligatorio               |
| Auditorías       | Opcional              | Obligatorias              |
| Training devs    | Opcional              | Obligatorio               |

## INDICADORES PARA CONTRATAR

| Indicador          | Verde   | Amarillo | Rojo    | Acción                    |
|--------------------|---------|----------|---------|---------------------------|
| Ratio QA:Dev       | 1:20-30 | 1:31-40  | >1:40   | Contratar                 |
| % tiempo gestión   | <30%    | 30-50%   | >50%    | Reestructurar o contratar |
| HUs sin revisar    | 0%      | <10%     | >10%    | Delegar IA o contratar    |
| Lead time HU→casos | <2 días | 2-5 días | >5 días | Optimizar IA o contratar  |
| Burnout QA         | <5/10   | 5-7/10   | >7/10   | Contratar urgente         |
| Incidentes prod    | Estable | +10-20%  | >+20%   | Mejorar o contratar       |
| Gates bypassed     | <5%     | 5-10%    | >10%    | Enforcement o contratar   |

Fórmula de decisión:

**CONTRATAR SI:**

(Ratio > 1:35)

**Y**

(% gestión > 40% O HUs sin revisar > 5%)

**Y**

(Burnout > 6/10 O Incidentes +15%)

---

VIABILIDAD SIN PRESUPUESTO

Respuesta honesta: SÍ, pero con concesiones importantes

✅ VIABLE: - Calidad en Po (críticos) al 100% - Quality Gates protegen todos - IA + Champions cubren básico P1/P2 - Escalable tecnológicamente

⚠️ SE SACRIFICA: - Cobertura activa P1/P2 (solo auditorías) - Profundidad análisis (IA + Champions, no QA Senior) - Tiempo respuesta (QA solo Po + alertas) - Innovación en QA (mantienen status quo)

🔴 NO VIABLE LARGO PLAZO: - Crecer a 200+ devs sin contratar → COLAPSO - Innovación en QA - Retener talento (burnout) - Competir con empresas que invierten más

Recomendación: Solución táctica, no estratégica. Presionar para contratar cuando sea posible.

---

ROADMAP 30/60/90 DÍAS

FASE 1: DÍAS 1-30 — DIAGNÓSTICO + BASELINE + QUICK WINS

Objetivo: Establecer baseline, demostrar quick wins visibles, generar buy-in organizacional.

*Semana 1-2: Discovery (Días 1-14)*

| Día | Actividad                          | Responsable        | Entregable        |
|-----|------------------------------------|--------------------|-------------------|
| 1-2 | Discovery técnico y organizacional | QA Lead + Engineer | Assessment report |
| 3-5 | Entrevistas 9 equipos              | QA Lead + Engineer | Matriz Po/P1/P2   |

| Día   | Actividad                   | Responsable        | Entregable                    |
|-------|-----------------------------|--------------------|-------------------------------|
| 6-7   | Ruta crítica + mapa riesgos | QA Lead + Product  | Top 10-15 flujos              |
| 8-10  | Quality Baseline (DoR/DoD)  | QA Lead            | DoR/DoD publicado             |
| 11-12 | Quick win: 3-5 flujos       | QA Engineer        | Automatización en CI/CD       |
| 13-14 | Dashboard v1 + presentación | QA Lead + Engineer | Dashboard + exec presentation |

*Semana 3-4: Agentes IA + Expansión (Días 15-30)*

| Día   | Actividad                             | Responsable             | Entregable                      |
|-------|---------------------------------------|-------------------------|---------------------------------|
| 15-17 | Implementar Agente 1 (Cobertura)      | QA Engineer             | Análisis automático funcionando |
| 18-20 | Implementar Agente 4 (Métricas) - MVP | QA Engineer             | Dashboard automático            |
| 21-23 | Automatizar 3 flujos adicionales      | QA Engineer + Champions | 6-8 flujos total                |
| 24-25 | Quality Gates (warning mode)          | QA Engineer + DevOps    | Pipeline con gates              |
| 26-27 | Training QA Champions                 | QA Lead                 | 4-9 Champions capacitados       |
| 28-30 | Retrospectiva + ajuste                | QA Lead + Stakeholders  | Plan 31-60 ajustado             |

*Métricas de Éxito (Día 30)*

| KPI                                | Baseline | Target          |
|------------------------------------|----------|-----------------|
| Flujos críticos automatizados      | 0/15     | 6-8/15 (40-53%) |
| Cobertura funcionalidades críticas | 39%      | 50%             |
| Defect leakage rate                | 18%      | <15%            |
| QA Champions capacitados           | 0        | 4-9             |
| Quality Gates activos              | No       | Warning mode    |
| Dashboard publicado                | No       | Sí              |
| Buy-in stakeholders                | ?        | 80%+            |

## FASE 2: DÍAS 31-60 — ESTABILIZACIÓN + AUTOMATIZACIÓN

Objetivo: Automatizar 95% flujos críticos, estabilizar pipelines, Quality Gates activos en PR.

### Semana 5-6 (Días 31-44)

| Actividad                         | Responsable             | Duración | Entregable               |
|-----------------------------------|-------------------------|----------|--------------------------|
| Automatizar 7-9 flujos restantes  | QA Engineer + Champions | 10 días  | 95% flujos automatizados |
| Implementar Agente 2 (Clean Code) | QA Engineer             | 4 días   | Code review automático   |
| Refactorizar pruebas flaky        | QA Engineer             | Continuo | Flakiness <5%            |
| Expandir cobertura API            | QA Engineer + Devs      | 10 días  | APIs críticas >90%       |

### Semana 7-8 (Días 45-60)

| Actividad                               | Responsable          | Duración | Entregable              |
|-----------------------------------------|----------------------|----------|-------------------------|
| Habilitar Quality Gates en PR (bloqueo) | QA Engineer + DevOps | 3 días   | Gates bloqueando PRs    |
| Training developers                     | QA Lead + Champions  | 5 días   | 30+ devs capacitados    |
| Estabilizar pipeline E2E                | QA Engineer          | 5 días   | Pass rate >95%, <15 min |

| Actividad               | Responsable          | Duración | Entregable           |
|-------------------------|----------------------|----------|----------------------|
| Performance smoke tests | QA Engineer + DevOps | 3 días   | Baseline establecido |
| Retrospectiva           | QA Lead              | 2 días   | Plan 61-90           |

*Métricas de Éxito (Día 60)*

| KPI                                | Día 30  | Target Día 60  |
|------------------------------------|---------|----------------|
| Flujos críticos automatizados      | 6-8/15  | 14-15/15 (95%) |
| Cobertura funcionalidades críticas | 50%     | 85%            |
| Defect leakage rate                | <15%    | <10%           |
| Quality Gates en PR                | Warning | Bloqueo activo |
| Pipeline pass rate                 | ?       | >95%           |
| Flakiness rate                     | ?       | <5%            |
| Developers capacitados             | 0       | 30+            |

FASE 3: DÍAS 61-90 — ESCALAMIENTO + CULTURA

Objetivo: Cultura Quality Engineering, ownership compartido, pipelines maduros.

*Semana 9-10 (Días 61-74)*

| Actividad                              | Responsable          | Duración | Entregable                  |
|----------------------------------------|----------------------|----------|-----------------------------|
| Implementar Agente 3 (Casos uso) - MVP | QA Engineer          | 7 días   | QA Copilot funcionando      |
| Shift-left: Calidad en refinamiento    | QA Lead + POs        | Continuo | Checklist refinamiento      |
| Developers ownership unit tests (80%)  | Tech Leads + QA      | Continuo | 80% coverage repos críticos |
| Quality Gates en deploy (bloqueo)      | QA Engineer + DevOps | 3 días   | Gates bloqueando deploys    |
| Expandir performance testing           | QA Engineer + DevOps | 7 días   | Performance suite CI/CD     |

Semana 11-12 (Días 75-90)

| Actividad                        | Responsable            | Duración | Entregable                  |
|----------------------------------|------------------------|----------|-----------------------------|
| Optimizar QA Copilot             | QA Engineer            | 5 días   | Prompts refinados           |
| Auditoría calidad organizacional | QA Lead                | 3 días   | Report madurez por equipo   |
| Retrospectiva organizacional     | QA Lead + Stakeholders | 2 días   | Lessons learned             |
| Documentación estrategia         | QA Lead                | 3 días   | Playbook QA ACME Corp       |
| Presentación ejecutiva final     | QA Lead                | 2 días   | Resultados + roadmap futuro |

Métricas de Éxito (Día 90)

| KPI                                | Día 60   | Target Día 90    | Mejora            |
|------------------------------------|----------|------------------|-------------------|
| Flujos críticos automatizados      | 14-15/15 | 15/15 (100%)     | ✓                 |
| Cobertura funcionalidades críticas | 85%      | 95%              | +144% vs baseline |
| Defect leakage rate                | <10%     | <5%              | -72% vs baseline  |
| Production incidents               | ?        | -50% vs baseline | -50%              |
| Deployment success rate            | ?        | >95%             | —                 |
| Quality Gates bypass rate          | ?        | <5%              | —                 |

|                               |        |                     |        |
|-------------------------------|--------|---------------------|--------|
| KPI                           | Día 60 | Target Día 90       | Mejora |
| Unit test coverage (críticos) | ?      | >80%                | —      |
| QA Copilot adoption           | 0%     | 60%+ HUs            | —      |
| Champions autónomos           | ?      | 80%+ sin escalation | —      |

POST-90 DÍAS: ROADMAP FUTURO (Días 91-180)

| Iniciativa              | Descripción                                    |
|-------------------------|------------------------------------------------|
| Escalar a P1            | Expandir cobertura de 4 Po → 7 equipos (Po+P1) |
| QA Copilot completo     | Generación código automatización               |
| Agente 5                | Si escalas sin presupuesto                     |
| Mobile testing          | Automatización mobile (si aplica)              |
| Chaos engineering       | Pruebas resiliencia                            |
| Certificación Champions | Programa formal                                |
| Fine-tuning IA          | Agentes aprenden patrones ACME                 |

QUICK WINS DETALLADOS (Ejemplo ACME Corp)

Quick Win 1: Automatización 3-5 flujos críticos (Días 11-12)

| Flujo                | Tipo        | Esfuerzo | Impacto              |
|----------------------|-------------|----------|----------------------|
| Login/Auth           | API + E2E   | 1 día    | Protege acceso total |
| Send message         | API + E2E   | 1 día    | Core negocio         |
| Process payment      | API         | 0.5 día  | Revenue              |
| Receive notification | Integration | 0.5 día  | Engagement           |
| Create group chat    | API + E2E   | 0.5 día  | Premium              |

Total: 3.5 días → 5 flujos protegidos en CI/CD

Resultado: 33% flujos críticos en 2 semanas

## RIESGOS Y CONTINGENCIAS

| Riesgo                   | Probabilidad | Plan                               |
|--------------------------|--------------|------------------------------------|
| No 6-8 flujos en 30 días | Media        | Priorizar top 5, diferir resto     |
| Resistencia Po           | Media        | Escalar, mostrar datos incidentes  |
| Agente 1 no funciona     | Baja         | Manual temporal, optimizar prompts |
| No encuentras Champions  | Media        | Tech Leads temporalmente           |
| Gates bloquean todo      | Alta         | Progresivo, ajustar umbrales       |
| Presupuesto IA mayor     | Baja         | Modelos más baratos                |

---

## GESTIÓN DEL CAMBIO

### Estrategia de Adopción Organizacional

Desafío: Resistencia moderada, diferentes niveles de madurez.

#### Fase 1: Crear Aliados (Días 1-30)

Acciones: - Identificar early adopters (futuros Champions) - Quick wins visibles para credibilidad - Comunicación frecuente de valor (dashboard, métricas) - Escuchar pain points reales

Resultado esperado: 4-9 Champions identificados, buy-in 80%+

---

#### Fase 2: Expandir Círculo (Días 31-60)

Acciones: - Training a developers (testing basics) - Casos de éxito de equipos early adopters - Soporte intensivo a Champions - Mostrar datos de mejora (defect leakage, incidentes)

Resultado esperado: 30+ developers capacitados, resistencia reducida

---

#### Fase 3: Normalizar (Días 61-90)

Acciones: - Quality como parte de cultura - Retrospectivas organizacionales - Reconocimiento a Champions y equipos - Documentación de estándares

Resultado esperado: Calidad compartida establecida, Champions autónomos 80%+

---

### Manejo de Resistencia

Señales de resistencia:

- “Ya hacemos suficientes pruebas”
- “QA ralentiza el desarrollo”
- “No tenemos tiempo para tests”
- “Los tests no sirven, siempre fallan”

Estrategia:

1. Escuchar: Pain points reales detrás de la resistencia
2. Empatizar: “Entiendo que el tiempo es limitado...”
3. Mostrar cómo QA facilita (no bloquea): Quality Gates automáticos, QA Copilot ahorra tiempo
4. Datos > opiniones: Mostrar incidentes históricos, impacto en clientes
5. Buscar aliados: Identificar developer que vea valor, convertir en Champion
6. Apoyo de management: Escalar si resistencia persiste en equipo Po

No hacer:

- Confrontar directamente
- Imponer procesos sin explicar por qué
- Ignorar feedback legítimo
- Usar autoridad sin datos

## RIESGOS Y MITIGACIONES

| #     | Riesgo                             | Prob  | Impacto | Mitigación                                                             | Owner       |
|-------|------------------------------------|-------|---------|------------------------------------------------------------------------|-------------|
| R-o 1 | Resistencia organizacional         | Media | Alto    | Champions, quick wins, datos, apoyo management                         | QA Lead     |
| R-o 2 | No lograr 95% cobertura en 90 días | Media | Medio   | Priorizar flujos críticos, ajustar expectativas, foco Po               | QA Lead     |
| R-o 3 | Burnout equipo QA (2 personas)     | Alta  | Crítico | Automatización, delegación Champions, contratar si posible             | QA Lead     |
| R-o 4 | Agentes IA no funcionan bien       | Baja  | Medio   | Fallback manual temporal, optimizar prompts, cambiar LLM               | QA Engineer |
| R-o 5 | Quality Gates bloquean delivery    | Alta  | Alto    | Configuración progresiva, ajustar umbrales, excepciones con aprobación | QA Engineer |
| R-o 6 | Champions no tienen tiempo         | Alta  | Alto    | Reconocimiento formal rol, % tiempo asignado oficialmente (20-30%)     | Managers    |
| R-o 7 | Presupuesto IA mayor a esperado    | Baja  | Bajo    | Optimizar uso, reducir invocaciones, modelos más baratos               | QA Engineer |
| R-o 8 | Pérdida de QA Champion             | Media | Medio   | Backup identificado, documentación, training nuevo Champion            | QA Lead     |

| #    | Riesgo                                | Prob | Impacto | Mitigación                                               | Owner   |
|------|---------------------------------------|------|---------|----------------------------------------------------------|---------|
| R-09 | Incidentes en P1/P2 (sin presupuesto) | Alta | Alto    | Auditorías, Quality Gates estrictos, alertas automáticas | QA Lead |
| R-10 | No hay presupuesto para contratar     | Alta | Alto    | Estrategia sin presupuesto (ver sección Escalabilidad)   | QA Lead |

---

## KPIS Y MÉTRICAS

### Métricas por Audiencia

#### *Para Dirección / Ejecutivos (mensual)*

- Production incidents trend: Cantidad y severidad (objetivo: -50% en 90 días)
  - Deployment success rate: % deploys sin rollback (objetivo: >95%)
  - Defect leakage rate: % defectos en prod vs pre-prod (objetivo: <5%)
  - Critical flows coverage: % flujos críticos protegidos (objetivo: 95%)
  - ROI de QA: (Costo incidentes evitados - Costo QA) / Costo QA
- 

#### *Para Engineering Managers (semanal)*

- Team quality score: Score consolidado por equipo (cobertura + defectos + gates)
  - Defect leakage rate por equipo
  - Automation coverage por equipo
  - Quality Gate pass rate por equipo
  - Technical debt de testing: Pruebas flaky, obsoletas, sin mantener
- 

#### *Para QA y Developers (diario)*

- Test pass rate: % pruebas que pasan en último run
- Flaky tests count: Cantidad de pruebas inestables
- Pipeline execution time: Tiempo total pipeline
- Coverage gaps: Funcionalidades sin cobertura
- Failed builds: Cantidad de builds fallidos hoy
- Open defects: Defectos abiertos por severidad

---

### Fórmulas Clave

Defect Leakage Rate = (Defectos prod / Total defectos) × 100

Critical Coverage = (Flujos críticos automatizados / Total críticos) × 100

Deployment Success = (Deploys sin rollback / Total deploys) × 100

Test Pass Rate = (Tests passed / Total tests executed) × 100

Flakiness Rate = (Tests flaky / Total tests) × 100

ROI QA = (Costo incidentes evitados - Costo QA) / Costo QA × 100

---

### Definición de cada KPI

#### *Defect Leakage Rate*

Definición: Porcentaje de defectos encontrados en producción vs total de defectos.

Fórmula: (Defectos encontrados en Prod / Total defectos encontrados) × 100

Fuente: Jira + Monitoring

Frecuencia: Semanal

Owner: QA Lead

Target: <5% (vs 18% baseline)

---

#### *Critical Flow Coverage*

Definición: Porcentaje de flujos críticos del negocio con automatización completa.

Fórmula: (Flujos críticos con automatización / Total flujos críticos) × 100

Fuente: Agente 1 (Cobertura) + Manual

Frecuencia: Semanal

Owner: QA Engineer

Target: 95%

---

### *Deployment Success Rate*

Definición: Porcentaje de deploys a producción exitosos (sin rollback).

Fórmula:  $(\text{Deploys exitosos} / \text{Total deploys}) \times 100$

Fuente: CI/CD + Monitoring

Frecuencia: Diaria

Owner: DevOps + QA Lead

Target: >95%

---

## ASSUMPTIONS Y OPEN QUESTIONS

### Assumptions Realizadas

1. Stack tecnológico es Node.js + AWS + Serverless
  2. Presupuesto para herramientas de IA existe (~\$17/mes inicial)
  3. Management apoya la estrategia de calidad
  4. Jira o similar está disponible para gestión
  5. CI/CD existe (Jenkins/GitLab/GitHub Actions)
  6. Champions pueden dedicar 20-30% tiempo a QA
  7. Documentación mínima de arquitectura existe
  8. Acceso a repos, pipelines, monitoring disponible
  9. Plataforma es web + API (mobile opcional)
  10. No hay compliance crítico (SOC2, GDPR) que bloquee inicio
- 

### Open Questions para Stakeholders

1. Presupuesto: ¿Cuál es el presupuesto máximo para contratar QA? (año 1, año 2)

2. Herramientas: ¿Presupuesto para herramientas adicionales? (test management ~\$50-100/usuario/mes)
  3. Stack actual: ¿Qué herramientas usan hoy? (Jira, CI/CD tool específico, monitoring)
  4. Definición incidentes: ¿Cuál es la definición interna de “production incident”?
  5. Compliance: ¿Hay requirements? (SOC2, GDPR, HIPAA, PCI-DSS)
  6. Mobile: ¿Está en scope o solo web/API?
  7. Time to market: ¿Cuál es la expectativa? (velocidad vs calidad)
  8. SLAs: ¿Hay SLAs definidos con clientes?
  9. Criticidad real: ¿Cuál es el costo promedio de un incidente en producción?
  10. Apoyo management: ¿Qué nivel de apoyo ejecutivo existe para transformación QA?
- 

## CONCLUSIÓN

### Resumen de la Estrategia

ACME Corp enfrenta desafíos típicos de organizaciones en crecimiento: regresiones frecuentes, baja cobertura (61% funcionalidades críticas sin automatización), incidentes en producción y un equipo QA inicial de solo 2 personas para 60 developers.

La estrategia propuesta transforma QA de un rol reactivo a Quality Engineering proactivo mediante 5 pilares fundamentales:

1. Ownership compartido: QA Champions + Developers como owners
  2. Testing pyramid optimizado: 60/30/10 + 95% cobertura crítica
  3. 4 agentes IA como multiplicador: Cobertura, Clean Code, Casos de uso, Métricas
  4. Quality Gates automáticos: Validación basada en riesgo en CI/CD
  5. Visibilidad continua: Dashboards y métricas por audiencia
- 

### Diferenciadores Clave

vs Enfoque Tradicional:

| Tradicional                     | Esta Estrategia                        |
|---------------------------------|----------------------------------------|
| QA como gatekeeper              | QA como facilitador                    |
| Contratar más QA                | Escalar con IA + Champions             |
| Automatizar todo                | Priorizar por riesgo (95% crítico)     |
| QA valida manualmente           | Quality Gates automáticos              |
| Casos de prueba manuales        | QA Copilot genera en minutos           |
| Cobertura uniforme              | Priorización radical Po/P1/P2          |
| Escalabilidad lineal (personas) | Escalabilidad exponencial (tecnología) |

---

### Impacto Esperado en 90 Días

| Métrica            | Baseline | Target | Mejora |
|--------------------|----------|--------|--------|
| Cobertura crítica  | 39%      | 95%    | +144%  |
| Defect leakage     | 18%      | <5%    | -72%   |
| Incidentes prod    | X        | -50%   | -50%   |
| Deployment success | Y%       | >95%   | —      |
| Flujos protegidos  | 0/15     | 15/15  | 100%   |

---

### Escalabilidad

El modelo escala de 60 a 150+ developers:

Con presupuesto: - 100 devs: +1 QA (total 3) - 150 devs: +3 QA (total 5) + estructura jerárquica

Sin presupuesto: - Priorización radical (solo 3-4 equipos Po) - IA como reemplazo parcial (no solo acelerador) - Champions autónomos + Lead Champions - Auditorías en lugar de cobertura continua - Viable corto/mediano plazo, riesgoso largo plazo

---

### Factores Críticos de Éxito

1. Apoyo de management: Transformación requiere sponsorship ejecutivo
  2. Quick wins visibles: Demostrar valor en primeros 30 días
  3. Quality Gates automáticos: Escalan sin límite, protegen sin intervención humana
  4. QA Champions comprometidos: 20-30% tiempo oficial asignado
  5. IA como multiplicador: Agentes bien implementados ahorran 70-80% tiempo
  6. Developers ownership: Unit tests no opcionales
  7. Métricas visibles: Dashboard desde día 14
  8. Gestión de resistencia: Datos > opiniones, escuchar pain points
- 

### Riesgos Principales

| Riesgo                     | Mitigación                                         |
|----------------------------|----------------------------------------------------|
| Burnout 2 QAs              | Automatización + delegación + contratar si posible |
| Resistencia organizacional | Champions + quick wins + datos + management        |
| Quality Gates bloquean     | Progresivo + ajuste umbrales                       |
| No hay presupuesto         | Estrategia sin presupuesto (radical pero viable)   |

---

### Próximos Pasos Inmediatos

Semana 1: 1. Aprobación de estrategia por stakeholders 2. Solicitar accesos (repos, Jira, CI/CD, monitoring) 3. Agendar entrevistas con 9 Tech Leads + POs 4. Iniciar diagnóstico técnico (assessment)

Semana 2: 5. Completar entrevistas y matriz de equipos 6. Identificar ruta crítica con Product 7. Preparar presentación día 8 (hallazgos + plan)

Día 14: 8. Presentar dashboard v1 + quick wins 9. Obtener buy-in para continuar fases 2 y 3

---

## Mensaje Final

Esta estrategia no es solo un plan técnico de automatización. Es una transformación cultural de cómo ACME Corp entiende y gestiona la calidad.

El éxito depende tanto de la ejecución técnica (agentes IA, Quality Gates, automatización) como de la adopción organizacional (Champions, ownership compartido, gestión del cambio).

Con 2 personas, 90 días y restricciones reales, la única forma de escalar es mediante tecnología (IA + automatización) y delegación (Champions + Developers). Este plan lo hace viable.

La calidad no es un departamento. Es una responsabilidad compartida.

---

## Comparación con Alternativas

### OPCIÓN A: Solo IA (estrategia actual)

- Costo 90 días: \$50
- Costo anual: ~\$200
- Capacidad: 2 personas + IA + Champions
- ROI: 75% ahorro tiempo diseño casos + 50% reducción incidentes

### OPCIÓN B: Contratar +1 QA (sin IA)

- Costo 90 días: \$15,000 - \$20,000 (salario 3 meses)
- Costo anual: \$60,000 - \$80,000
- Capacidad: 3 personas sin multiplicador IA
- ROI: Cobertura mejorada pero sin automatización de diseño

### OPCIÓN C: IA + Contratar (combinado)

- Costo 90 días: \$15,050 - \$20,050
  - Costo anual: \$60,200 - \$80,200
  - Capacidad: 3 personas + IA + Champions
  - ROI: Máximo (capacidad humana + multiplicador tecnológico)
-

ROI Estimado

Ahorro de Tiempo (QA Copilot)

Sin IA: - 30-40 HUs/sprint - 3 horas/HU diseño de casos - Total: 90-120 horas/sprint - Con 2 personas: IMPOSIBLE

Con IA: - 30-40 HUs/sprint - 45 min/HU (IA genera, QA revisa) - Total: 22-30 horas/sprint - Con 2 personas: VIABLE

Ahorro: 70-90 horas/sprint = \$3,500-4,500/mes (valor tiempo QA Senior @ \$50/hora)

ROI primer mes:  $(\$3,500 \text{ ahorro} - \$17 \text{ costo}) / \$17 = 20,482\%$

---

Ahorro en Incidentes Evitados

Baseline: - Defect leakage: 18% - ~10 incidentes/mes en producción - Costo promedio incidente: \$5,000 (estimado: downtime + fix + reputación) - Costo mensual incidentes: \$50,000

Con estrategia (90 días): - Defect leakage: <5% (objetivo) - ~5 incidentes/mes (50% reducción) - Costo mensual incidentes: \$25,000

Ahorro mensual: \$25,000

ROI anual:  $(\$25,000 \times 12 \text{ meses} - \$200 \text{ costo IA}) / \$200 = 149,900\%$

---

Inversión Total vs Beneficio

| Concepto                   | Costo/Ahorro Anual                                           |
|----------------------------|--------------------------------------------------------------|
| Inversión IA               | -\$200                                                       |
| Ahorro tiempo QA           | +\$42,000 - \$54,000                                         |
| Ahorro incidentes evitados | +300,000(50  ** Mejoratimetomarket **   +X (no cuantificado) |
| ROI NETO                   | +\$341,800 - \$353,800                                       |

Retorno: 170,900% - 176,900% sobre inversión en IA

---

### Conclusión Financiera

La inversión en IA de \$50 en 90 días (\$200 anual) genera:

1.  Ahorro directo de tiempo QA: \$42K-54K/año
2.  Ahorro en incidentes evitados: \$300K/año
3.  Escalabilidad sin contratar: Invaluable
4.  ROI >170,000%: Una de las inversiones más rentables posibles

Recomendación ejecutiva: Aprobar inversión inmediatamente. El costo es negligible comparado con el retorno.

---